

Code Assessment of the PT Oracle Smart Contracts

25 November, 2025

Produced for



by



Contents

1	Executive Summary	3
2	Assessment Overview	5
3	Limitations and use of report	8
4	Terminology	9
5	Open Findings	10
6	Open Questions	11
7	Informational	12
8	Notes	15

1 Executive Summary

Dear Curve Team,

Thank you for trusting us to help Curve with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of PT Oracle according to [Scope](#) to support you in forming an opinion on their security risks.

Curve implements an oracle for pricing Pendle Principal Tokens (PT) by applying a time-based linear discount to the underlying asset prices.

The most critical subjects covered in our audit are access control, functional correctness, and precision of arithmetic operations. Functional correctness is improvable due to [Incorrect Discount Parameters Check](#) when updating the discount parameters, which could lead to the oracle being unusable. Security regarding access control and precision of arithmetic operations is high.

Other general subjects covered include gas efficiency and integration in Curve. Security regarding all the aforementioned subjects is high.

The note [Usage of the oracle in Curve LlamaLend markets](#) describes important economic implications of using this oracle in a Curve LlamaLend AMM that are out of scope for this review.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	1
• Pending	1
Low -Severity Findings	0

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the PT Oracle repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	19 September 2025	5e0615e9af72d675ac840d2c729adfed14676027	Initial Version

For the Vyper smart contracts, the compiler version 0.4.3 was chosen.

The only file that was in scope for this review is:

- `contract/PTOracle.vy`

2.1.1 Excluded from scope

The underlying oracle, Pendle PT token, tests, deployment scripts, and documentation files were excluded from the scope of this review.

2.2 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Curve offers a discounted price feed for a Pendle Principal Token (PT) relative to its underlying asset. The contract computes the discounted PT price based on a linear discount model parameterized by a slope and intercept, with respect to the PT's time to maturity. It integrates with an external underlying price oracle and enforces access-controlled updates to its discount parameters.

The `PTOracle` contract is the sole contract in scope. It is non-reentrant by default and inherits role-based access control functionality from the imported `snekmate's access_control` module.

Its primary purpose is to provide a PT price that reflects both the underlying asset's spot price and a time-decaying linear discount, suitable for use in Curve LlamaLend when needing a fair valuation of a non-matured PT.

Price Computation

- The oracle queries the configured underlying oracle for the current price of the underlying asset.
- If the PT has expired (`block.timestamp >= pt_expiry`), the oracle simply returns the underlying price.
- Otherwise, it applies a linear discount model defined as:

$$discount = \frac{slope \times time_to_maturity_years}{10^{18}} + intercept$$

The discount reduces linearly with time to maturity and is bounded to ensure it does not exceed full price.

- The discounted price is then:

$$price = \frac{underlying_price \times (10^{18} - discount)}{10^{18}}$$

The oracle exposes two pricing functions, according to the interface expected by LlamaLend's AMM:

- `price()` a view function that calculates and returns the current price on demand.
- `price_w()` a state-changing function that updates the cached price and timestamp to improve gas efficiency.

Discount Parameter Management

- The linear discount parameters, `slope` and `intercept`, can be updated by accounts holding the **MANAGER_ROLE**.
- Updates are rate-limited by `min_update_interval` to mitigate rapid parameter manipulation.
- To guard against abrupt market shifts, the contract enforces maximum per-update change limits: `max_slope_change` and `max_intercept_change`.
- A helper function, `set_slope_from_apy()`, allows managers to derive the slope from a desired annualized yield (APY), automatically setting the intercept to zero.

Administration

All privileged operations are controlled by roles granted at deployment:

- **ADMIN_ROLE**: overall administrative authority; the role admin for all subordinate roles.
- **MANAGER_ROLE**: permitted to adjust discount parameters subject to rate limits.
- **PARAMETER_ADMIN_ROLE**: permitted to adjust update intervals and per-update change limits.

2.3 Trust Model

The contract is not upgradeable by design.

Economical Assumptions

- This system assumes that a linear relationship between time to maturity and discount is a reasonable approximation for the PT's fair value, this assumption, along with the specific parameter values, and consequently the accuracy of the price provided by the oracle is considered out of scope for this review.
- This oracle is intended to be used as a collateral price oracle for Curve LlamaLend markets. The note [Usage of the oracle in Curve LlamaLend markets](#) describes important economic implications of this design choice that are out of scope for this review.

External Dependencies

- **Underlying Oracle**: Assumed to provide an accurate and up-to-date spot price for the underlying asset, expected to be manipulation-resistant and never reverting.
- **Pendle PT**: Assumed to comply with the interface and return a fixed, immutable expiry timestamp.

Roles

The contract uses a strict role-based access control scheme:

DRAFT

- **ADMIN_ROLE** (fully trusted):

- Can grant or revoke the other two roles.
- Risk: A compromised admin can replace managers or parameter admins and manipulate discount parameters maliciously.

- **MANAGER_ROLE** (fully trusted):

- Can adjust discount parameters within the configured limits and intervals.
- Risk: Misconfiguration or collusion may lead to unrealistic pricing or price jumps, potentially harming integrators relying on the oracle.

- **PARAMETER_ADMIN_ROLE** (fully trusted):

- Can adjust operational safety limits such as rate limits and change caps.
- Risk: Setting limits too high or too low may disrupt timely parameter updates or allow excessive market manipulation.

- **End Users and Integrators**

- Consumers of the oracle price (e.g., lending protocols, AMMs) are assumed to trust the oracle's parameter configuration and the underlying oracle.
- End users have no special privileges, they can call public view functions.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe our findings. The findings are split into these different categories:

- **Correctness**: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	1
• Incorrect Discount Parameters Check	
Low -Severity Findings	0

5.1 Incorrect Discount Parameters Check

Correctness **Medium** **Version 1**

CS-CURVE-PT-ORACLE-001

In `_update_discount_params()`, it is essential that the new slope and intercept values are set in such a way that discount is always smaller than or equal to `1e18`, as otherwise, `_calculate_price()` would revert due to an underflow when computing `DISCOUNT_PRECISION - discount`.

In practice, because `time_to_maturity_years` is always decreasing over time, it is sufficient to ensure that the initial discount at the time of calling `_update_discount_params()` is less than or equal to `1e18`:

```
# Linear discount with 1e18 precision: discount = (slope * time_to_maturity_years) / 1e18 + intercept
discount: uint256 = (
    self.slope * time_to_maturity_years
) // DISCOUNT_PRECISION + self.intercept
assert discount <= DISCOUNT_PRECISION, "discount exceeds precision"
```

However, this check relies on `self.slope` and `self.intercept` that have not yet been updated to the new values. Therefore, it is possible for a malicious or careless Manager to set new discount parameters that would make the oracle consistently reverting, effectively making it unusable.

6 Open Questions

Here, we list open questions that came up during the assessment and that we would like to clarify to ensure that no important information is missing.

6.1 Design Decision

Open Question Version 1

`set_linear_discount` and `set_slope_from_apy` both set the discount parameters.

1. Why are both needed? The intercept and slope can easily be calculated off-chain from the APY or vice versa.
2. Why does one function allow setting the intercept and the other always sets it to 0?
3. What are the practical scenarios where the intercept must be set to non-zero values?

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Code Duplication

Informational **Version 1**

CS-CURVE-PT-ORACLE-002

`_update_discount_params()` and `_calculate_price()` both contain the same logic to compute the discount, which is currently duplicated. This logic could be refactored into a separate internal function to reduce code duplication and improve maintainability.

7.2 Gas Savings

Informational **Version 1**

CS-CURVE-PT-ORACLE-003

The following gas optimizations were identified in the codebase:

1. `pt` could be made immutable as it is set only once in the constructor.
2. `underlying_oracle` could be made immutable as it is set only once in the constructor.
3. In `__init__()`, when logging the `PriceUpdated`, the storage load of `self.last_price` could be avoided as the value was stored just before.
4. In `_update_discount_params`, `self.slope` and `self.intercept` are loaded from storage multiple times. Caching them in local variables would save gas.

7.3 Incorrect Comments

Informational **Version 1**

CS-CURVE-PT-ORACLE-004

1. The following comment is incorrect as `max_slope_change` and `max_intercept_change` prevent any change to their respective parameters if set to zero:

```
# Limit variables for slope and intercept changes (0 = no limit)
```

2. The following comment is misleading as this variable is also used in `set_slope_from_apy()`:

```
# Rate limiting for set_linear_discount
last_discount_update: public(uint256)
```

7.4 Indexed Event

Informational **Version 1**

CS-CURVE-PT-ORACLE-005

The `PriceUpdated` event indexes the new price. This is uncommon as the price is quasi-continuous and not discrete.

7.5 Missing NatSpec Comments

Informational **Version 1**

CS-CURVE-PT-ORACLE-006

Except for `set_slope_from_apy()`, all external functions lack NatSpec comments describing their purpose, parameters and return values. While the function names are mostly self-explanatory, adding NatSpec comments would enhance code readability and maintainability, especially for new developers or auditors unfamiliar with the codebase.

7.6 Out of Date Price in `price_w` Function

Informational **Version 1**

CS-CURVE-PT-ORACLE-009

While the function `price` always return up-to-date prices with respect to the underlying price oracle, `price_w` might return a cached price if:

- Expiry is not reached yet
- and the last call to `price_w` was in the same block as the current call

In such case, the function will return `self.last_price`. This could be problematic if the underlying price oracle can have multiple updates in the same block, as the PT oracle would not reflect the latest of those updates.

7.7 Price Bump

Informational **Version 1**

CS-CURVE-PT-ORACLE-007

Given a fixed underlying price, the PT price returned by this oracle might have sudden jumps at certain times:

1. Right after the PT expiry, if `intercept` is non-zero, the price jumps from a discounted value to the full underlying price
2. When the `slope` and `intercept` are set

The Manager of the contract should ensure that the parameters are set in a way that avoids unexpected price jumps that could hurt users of the oracle.

7.8 Stale `last_price` After Expiry

Informational Version 1

CS-CURVE-PT-ORACLE-008

After the PT expiry, the price returned by `price_w()` is no longer stored in `last_price`. Integrators relying on the stored price may be affected if they do not switch to calling the view function `price()` instead.

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Usage of the Oracle in Curve LlamaLend Markets

Note **Version 1**

This oracle is intended to be used as a collateral price oracle for Curve LlamaLend markets. By design, the price of this oracle does not reflect the actual market price of the PT token, but rather an estimate of it based on a linear function. This design choice brings non-trivial economic implications when using this oracle in a Curve LlamaLend AMM, as it can lead to situations where the oracle price diverges significantly from the market price, potentially causing unexpected liquidations or trading opportunities. These implications are out of scope for this review, but we recommend that the Curve team carefully consider them before deploying this oracle in production.

8.2 ADMIN_ROLE Cannot Be Granted or Revoked

Note **Version 1**

The ADMIN_ROLE is assigned at deployment, however as no role admin is given to that role, it cannot be granted or revoked later, only the current holder can renounce it.